

Real Estate — Companies, Properties & Agents

Goal: Add company profiles + their properties + agent section to your real estate website (like MagicBricks). This document contains a complete spec: database schema (MySQL), SQL migrations, seed examples, REST API design, frontend component list (React), admin flows, search & filtering, validations, and implementation notes.

1) High-level entities

- **Company** — real estate developer / agency (profile, logo, address, contact, verification)
 - **Agent** — individual broker or salesperson (linked to company or independent)
 - **Property** — listing (rental/sale) with details, features, images, geolocation
 - **PropertyImages** — multiple images per property
 - **PropertyFeatures / Amenities** — standardized list (e.g., parking, lift)
 - **Location** — city / locality / lat / lng
 - **PropertyType** — apartment/villa/studio
 - **Listing** — link between Property, Agent, Company + listing-specific info, status
 - **PriceHistory** — history of price changes (optional)
-

2) MySQL schema (DDL)

```
-- companies
CREATE TABLE companies (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL UNIQUE,
  logo VARCHAR(512),
  website VARCHAR(512),
  phone VARCHAR(50),
  email VARCHAR(255),
  description TEXT,
  address TEXT,
  city VARCHAR(128),
  state VARCHAR(128),
  country VARCHAR(64) DEFAULT 'India',
  pincode VARCHAR(20),
  verified TINYINT(1) DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- agents
```

```

CREATE TABLE agents (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_id BIGINT NULL,
  first_name VARCHAR(128) NOT NULL,
  last_name VARCHAR(128),
  slug VARCHAR(255) NOT NULL UNIQUE,
  profile_image VARCHAR(512),
  phone VARCHAR(50),
  email VARCHAR(255),
  bio TEXT,
  is_active TINYINT(1) DEFAULT 1,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE SET NULL
);

-- property_types
CREATE TABLE property_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(64) NOT NULL,
  slug VARCHAR(64) NOT NULL UNIQUE
);

-- locations
CREATE TABLE locations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  city VARCHAR(128) NOT NULL,
  locality VARCHAR(255),
  latitude DECIMAL(10,7),
  longitude DECIMAL(10,7),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- properties
CREATE TABLE properties (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_id BIGINT NULL,
  agent_id BIGINT NULL,
  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL UNIQUE,
  description TEXT,
  property_type_id INT,
  status ENUM('available', 'sold', 'rented', 'under_construction') DEFAULT
'available',
  price DECIMAL(18,2) NOT NULL,
  price_unit ENUM('INR', 'USD') DEFAULT 'INR',
  price_negotiable TINYINT(1) DEFAULT 0,
  bedrooms INT DEFAULT 0,

```

```

    bathrooms INT DEFAULT 0,
    builtup_area DECIMAL(10,2),
    carpet_area DECIMAL(10,2),
    floor_number INT,
    total_floors INT,
    furnishing ENUM('furnished','semi-furnished','unfurnished') DEFAULT
'unfurnished',
    facing VARCHAR(64),
    location_id INT,
    address TEXT,
    is_featured TINYINT(1) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (company_id) REFERENCES companies(id) ON DELETE SET NULL,
    FOREIGN KEY (agent_id) REFERENCES agents(id) ON DELETE SET NULL,
    FOREIGN KEY (property_type_id) REFERENCES property_types(id),
    FOREIGN KEY (location_id) REFERENCES locations(id)
);

-- property_images
CREATE TABLE property_images (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    property_id BIGINT NOT NULL,
    url VARCHAR(1024) NOT NULL,
    alt_text VARCHAR(255),
    is_primary TINYINT(1) DEFAULT 0,
    sort_order INT DEFAULT 0,
    FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
);

-- amenities (master list)
CREATE TABLE amenities (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(128) NOT NULL UNIQUE
);

-- property_amenities (many-to-many)
CREATE TABLE property_amenities (
    property_id BIGINT NOT NULL,
    amenity_id INT NOT NULL,
    PRIMARY KEY (property_id, amenity_id),
    FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE,
    FOREIGN KEY (amenity_id) REFERENCES amenities(id) ON DELETE CASCADE
);

-- price_history (optional)
CREATE TABLE price_history (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

```

```

    property_id BIGINT NOT NULL,
    old_price DECIMAL(18,2) NOT NULL,
    new_price DECIMAL(18,2) NOT NULL,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    changed_by BIGINT NULL
);

-- saved_searches / favourites (optional)
CREATE TABLE favourites (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    property_id BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

3) Seed examples (small)

```

INSERT INTO property_types (name, slug) VALUES
('Apartment', 'apartment'),
('Villa', 'villa'),
('Studio', 'studio');

INSERT INTO companies (name, slug, phone, email, city) VALUES
('Ruchi Lifescape', 'ruchi-lifescape',
'9876543210', 'info@ruchilife.com', 'Bhopal');

INSERT INTO agents (company_id, first_name, last_name, slug, phone, email)
VALUES
(1, 'Ruchi', 'Sales', 'ruchi-sales', '9800000000', 'agent@ruchilife.com');

INSERT INTO locations (city, locality, latitude, longitude) VALUES
('Bhopal', 'Zone 2 MP Nagar', 23.2599, 77.4126);

INSERT INTO properties (company_id, agent_id, title, slug, description,
property_type_id, price, bedrooms, bathrooms, builtup_area, location_id,
address, is_featured) VALUES
(1, 1, '3 BHK Luxury Apartment in MP Nagar', '3-bhk-mp-nagar', 'Spacious 3 BHK with
balcony and parking', 1, 12500000, 3, 3, 1450, 1, 'Plot no - 170 Near Indian Bank Zone
2 MP Nagar', 1);

INSERT INTO property_images (property_id, url, alt_text, is_primary) VALUES
(1, '/uploads/prop_1_1.jpg', 'Living Room', 1),
(1, '/uploads/prop_1_2.jpg', 'Bedroom', 0);

```

4) REST API design (suggested)

Auth - POST /api/auth/login — returns JWT - POST /api/auth/register — optional (for agents/admins)

Companies - GET /api/companies — list (with query ?city= & ?verified=) - POST /api/companies — create (admin) - GET /api/companies/:id — details (include properties count) - PUT /api/companies/:id — update - DELETE /api/companies/:id — delete

Agents - GET /api/agents — list (filter by company_id, city) - POST /api/agents — create - GET /api/agents/:id — details (include properties) - PUT /api/agents/:id — update - DELETE /api/agents/:id — delete

Properties - GET /api/properties — search endpoint (see filters below) - POST /api/properties — create listing (agent/company) - GET /api/properties/:id — property detail (include images, amenities, company, agent) - PUT /api/properties/:id — update - DELETE /api/properties/:id — delete - POST /api/properties/:id/images — upload image - DELETE /api/properties/:id/images/:image_id — delete image

Search params for GET /api/properties - q (keyword), city, locality, property_type_id, min_price, max_price, bedrooms, bathrooms, furnishing, is_featured, sort (price_asc, price_desc, newest), page, per_page, lat, lng, radius_km

Example: GET /api/properties?
q=3+BHK&city=Bhopal&min_price=5000000&max_price=20000000&page=1

Response structure (property detail)

```
{
  "id": 1,
  "title": "3 BHK Luxury Apartment in MP Nagar",
  "slug": "3-bhk-mp-nagar",
  "price": 12500000,
  "price_unit": "INR",
  "bedrooms": 3,
  "bathrooms": 3,
  "builtup_area": 1450,
  "furnishing": "semi-furnished",
  "status": "available",
  "location": {"city": "Bhopal", "locality": "Zone 2 MP Nagar", "latitude": 23.2599, "longitude": 77.4126},
  "company": {"id": 1, "name": "Ruchi Lifescape", "slug": "ruchi-lifescape"},
  "agent": {"id": 1, "first_name": "Ruchi", "last_name": "Sales", "phone": "9800000000"},
  "images": [{"id": 11, "url": "/uploads/prop_1_1.jpg", "is_primary": true}],
  "amenities": ["Lift", "Parking"],
}
```

5) Frontend components (React + Tailwind suggested)

Pages - Companies directory (/companies) — list of companies with logo, location, verified badge, bright CTA: View Listings - Company detail (/companies/:slug) — company overview, contact, team, all listings, reviews - Agents directory (/agents) — filters by city / company - Agent profile (/agents/:slug) — contact, bio, properties - Properties listing (/properties) — search/filter page - Property detail (/properties/:slug) — gallery, details, contact agent, schedule visit - Admin dashboard — manage companies, agents, properties, uploads

Reusable components - `PropertyCard` — image, price, location, quick details - `CompanyCard` — logo, name, verified, short stats - `AgentCard` — avatar, name, company, phone - `SearchBar` + `FiltersPanel` — filters (range sliders for price, dropdowns) - `ImageCarousel` — property gallery (supports lazy-loading)

6) Admin flows & permissions

- Roles: `admin`, `company_admin`, `agent`
- `admin` can manage everything
- `company_admin` can manage their company profile and properties & agents for that company
- `agent` can manage own listings only

Admin actions: - Bulk import properties via CSV (map columns to fields) - Approve/verify companies & agents - Feature/unfeature properties - Export leads and enquiries as CSV

7) Search, performance & indexing

- Add indexes for common queries: `properties(price)`, `properties(location_id)`, `properties(property_type_id)`, `properties(bedrooms)`
- Add FULLTEXT index on `properties(title, description)` for keyword searches.
- For advanced filtering & geo queries use Elasticsearch or Algolia; otherwise MySQL with spatial index (POINT) and Haversine formula works.
- Cache frequent searches on server-side (Redis) with small TTL.

SQL examples:

```
ALTER TABLE properties ADD FULLTEXT idx_ft_title_desc (title, description);
ALTER TABLE locations ADD COLUMN coordinates POINT; -- and set SRID if needed
CREATE INDEX idx_prop_price ON properties(price);
```

8) Image handling & uploads

- Store images on S3 (or your server storage) — keep original + optimized web-sized versions
 - Store URLs in `property_images.url`
 - Provide automatic thumbnails & progressive loading
 - Validate uploads: max 10 images per property, max 5MB each, allowed types jpg/png/webp
-

9) Validation rules (sample)

- `Company.name` — required, 2..255 chars
 - `Agent.email` — valid email
 - `Property.title` — required, 6..255 chars
 - `Property.price` — required, positive number
 - `Property.slug` — unique, generated from title
 - `Location.latitude/longitude` — if present valid decimal ranges
-

10) Notifications & leads

- When user submits `Contact Agent` or `Schedule Visit` -> create `leads` table and send email + SMS to agent & company. Store lead status.

Sample leads table:

```
CREATE TABLE leads (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  property_id BIGINT,  
  user_name VARCHAR(255),  
  user_email VARCHAR(255),  
  user_phone VARCHAR(50),  
  message TEXT,  
  status ENUM('new', 'contacted', 'closed') DEFAULT 'new',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

11) Example: Property Create payload (frontend -> backend)

```
{  
  "title": "3 BHK Luxury Apartment in MP Nagar",  
  "description": "Spacious 3 BHK...",  
  "company_id": 1,
```

```
"agent_id":1,
"property_type_id":1,
"price":12500000,
"bedrooms":3,
"bathrooms":3,
"builtup_area":1450,
"location": {"city":"Bhopal","locality":"MP Nagar","latitude":
23.2599,"longitude":77.4126},
"amenity_ids":[1,3,4]
}
```

12) SEO & Schema.org

- Add meta title/description to property and company pages.
- Add JSON-LD `RealEstateAgent` and `Offer` schema for properties to improve rich snippets.

Sample JSON-LD (property page):

```
{
  "@context":"https://schema.org",
  "@type":"Offer",
  "itemOffered":{
    "@type":"Apartment",
    "name":"3 BHK Luxury Apartment in MP Nagar"
  },
  "price":"12500000",
  "priceCurrency":"INR",
  "seller":{
    "@type":"RealEstateAgent",
    "name":"Ruchi Lifescape"
  }
}
```

13) UX suggestions (to match MagicBricks feel)

- Prominent search bar on top with quick filters (city/locality, buy/rent toggle, budget)
- Listing grid + list toggles, map view with clustered markers
- Company and Agent badges (verified, top-seller)
- Chat/WhatsApp contact button on property page
- Quick enquiry modal that collects name/phone/email and sends lead

14) Implementation roadmap (sprints)

Sprint 1 (1-2 weeks) - DB migrations + seed essentials (companies, agents, property types) - API endpoints for CRUD (companies, agents, properties) - Basic property listing + detail pages

Sprint 2 (1 week) - Image upload flow, amenities, location - Company and agent pages, link listings - Admin role + simple UI to manage

Sprint 3 (1 week) - Search filters, fulltext search, pagination - Geo search and map view - Lead capture & email notifications

Sprint 4 (optional) - Performance tuning, cache, advanced search (Elasticsearch) - CSV import/export, analytics dashboard

15) Helpful code snippets

Generate slug (node / JS)

```
function slugify(text) {
  return text.toString().toLowerCase()
    .replace(/\s+/g, '-')
    .replace(/[\^a-z0-9\-\]/g, '')
    .replace(/\-+/g, '-')
    .replace(/\-+|\/+$/g, '');
}
```

Haversine example (MySQL) — properties within radius (km)

```
SELECT p.*, (6371 * acos(cos(radians(?lat?)) * cos(radians(l.latitude)) *
cos(radians(l.longitude) - radians(?lng?)) + sin(radians(?lat?)) *
sin(radians(l.latitude)))) AS distance_km
FROM properties p
JOIN locations l ON p.location_id = l.id
HAVING distance_km <= ?radius_km?
ORDER BY distance_km
LIMIT 0, 50;
```

16) Next steps I can do for you (pick any and I will create directly):

- Generate full set of SQL migration files (for a specific framework: Laravel / Node + Knex / Django)

- Create REST API code stubs (Express + Sequelize) + authentication
 - Build React components (complete code file for property card, listing page, property detail)
 - Prepare Admin panel wireframes & UI copy
-

If you want, tell me which stack you prefer (Laravel, Node/Express, Django, or PHP CodeIgniter) and I will generate code/migrations and a couple of frontend components right away.